When Software Fabricates Assurance: Seven Design Principles for Trustworthy Governance, Risk and Compliance Systems

Name of the Track: Track 1 – Information Systems: Theory and Practice (Digital Ethics and Policy: Regulation and Compliance)

Type of Submission: Ongoing research (design science research; artifact and technical evaluation completed, naturalistic evaluation pending)

Abstract

1. Purpose

Boards, auditors and regulators rely on governance, risk and compliance (GRC) information systems for evidence that risks are identified, rated, approved and controlled. Research on these systems has examined their adoption and value. We found no study asking whether the evidence they present can itself be trusted. A code and runtime audit of an existing risk management application found that the software invented risk ratings, discarded approvals, accepted unauthenticated authority and reported failed database writes as successes. This study asks which design failures compromise the integrity of assurance evidence. It also asks which design principles prevent them, and how well an instantiation of those principles performs against the original system.

2. Design/Methodology/Approach

The study follows design science research. The audit produced a taxonomy of seven failure modes. Each was matched to a design principle grounded in established theory: integrity models, fail-safe defaults, complete mediation, separation of duties, tamper-evident logging and the critique of ritualised assurance. The principles were instantiated in a new system, ERMA-GRC. Both systems were then run against identical stored data in 23 scenarios, whose protocol was time-stamped in the project repository before execution. Eighteen scenarios derive from the audited failures, four test the principles beyond them, and one was expected to fail. Every deviation from the protocol is reported.

3. Findings

The original system prevented the failure in 4 of the 18 audit-derived scenarios; ERMA-GRC prevented it in all 18. ERMA-GRC accepted no unauthenticated writes, rejected self-approval, recorded audit entries only under the signed-in identity, detected an edited audit entry and returned errors instead of false successes. It also prevented 3 of the 4 failures in the scenarios beyond the audit, including a race between two simultaneous approval decisions, and every stored-state outcome was read from the database stand-in, not from what either system reported. Two failures of the new design are reported with equal weight. First, a key risk indicator's status was computed correctly in the interface but could be declared by a client calling the server directly. Second, an insider holding the audit key could rewrite the evidence trail undetected after a restart. The principles also depend on one another: separation of duties relies on the integrity of the audit trail, and a rule enforced only in the client offers no assurance.

4. Originality/Value

The study treats assurance integrity as a design property of GRC information systems rather than an organisational outcome. It contributes three things: a failure taxonomy derived from a

real system, seven design principles with two testable propositions toward a nascent design theory, and a reusable scenario protocol. Auditors and buyers can use the principles as a review checklist. The documented boundaries point to future work: externally anchored audit trails, server-side computation of every assurance metric, and evaluation with practitioners.

Keywords

assurance integrity; governance, risk and compliance; design science research; enterprise risk management; audit trail; separation of duties